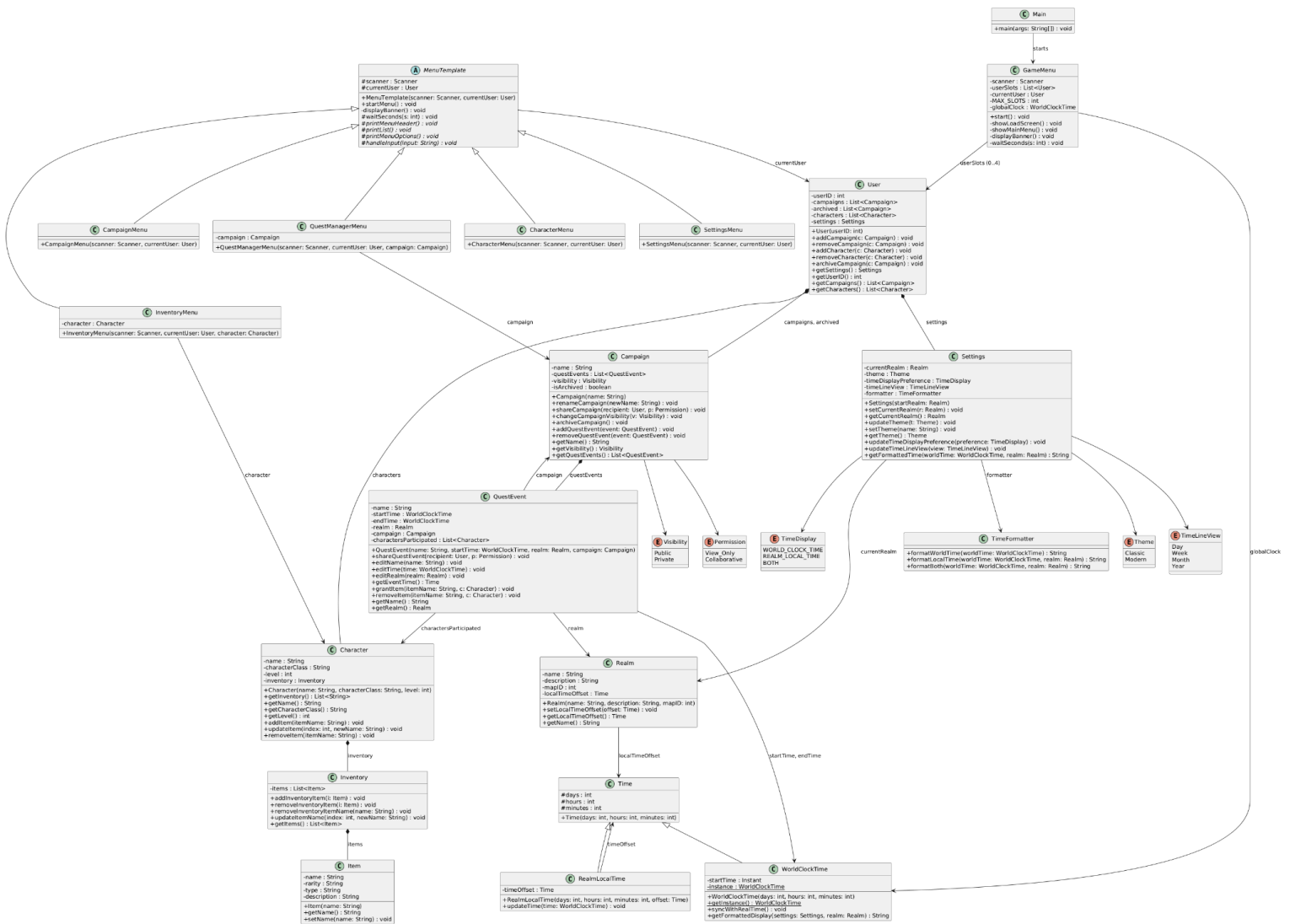


GuildQuest Documentation

Github Repo: <https://github.com/evanstew1/Inf122GroupProject>

UML Diagram	2
Testing	4
Retrospective	5
AI Use	6
Justification of Quality (Patterns + Smells)	7
Reuse	8

UML Diagram



Zoom into see full detail:

UML Description:

Main: Entry point of the game and it creates an instance of GameMenu to start the game

GameMenu: Acts as the central hub for the game. It handles the initial loading screen and routes users to sub-menus

MenuTemplate: An Abstract base class that establishes a standardized loop and layout for all UI menus. It also displays a banner, header, as well as menu specific options and more.

CampaignMenu: This is the interface for managing users' campaigns, such as adding them, naming them, deleting them, and selecting them.

CharacterMenu: This is the interface for managing users characters, such as adding them, naming/choosing classes for them, deleting them, and selecting them.

InventoryMenu: This is the interface for managing a characters inventory, such as adding items, removing items, or updating items

QuestManagerMenu: This is the interface for managing quest events, such as adding, viewing, or deleting a quest event

SettingsMenu: This is the interface for updating the settings, it allows the user to change how time is displayed as well as update their current realm or the application theme.

User: Represents a player profile, it holds a unique ID, a settings object as well as a list of their active campaigns, archived campaigns, and created characters.

Campaign: Represents a specific storyline within the game. It contains a name, visibility, archived status, as well as a list of quest events.

Inventory: A container class that manages a list of Item objects for a character. It provides the logic to add remove and update items by name/index

Item: A data structure that stores details about an item like rarity, type, and description of one.

QuestEvent: This is an event or mission within a campaign. The questevent class holds the quests name, realm, time in the realm, as well as the ability to grant/remove items from characters.

Realm: A representation of a specific in game location. The realm class stores the name, id, description, as well as the local time offset of the realm.

Settings: Stores what realm, ui theme, and time display preference the user has selected.

Time: Stores days, hours and minutes

WorldClockTime: A singleton class that extends time that represents the global time within the game universe

RealmLocalTime: Extension of time to represent the time in a specific realm and calculates the local time based off of the realms offset and the WorldClockTime

TimeFormatter: A utility class that converts WorldClockTime and RealmLocalTime and outputs them in a string.

Testing

Testing for the 2 Mini Games

1. Create A User (type **5** then enter)
2. Create a second user (type **5** then enter)
3. Open the User (type **1** then enter)
4. Open Mini Adventure (Type **4** then enter)
5. Open Mini Game 1 (Type **1** then enter)
6. Select user for player 2 (type **2** then enter)
7. Create a character or pick created 1 (Type **1** then enter)
8. Enter Character Name and enter Character Class
9. **Repeat steps 7 & 8** to make User 2's character
10. Press **A** to Attack, do this until the game is won or time runs out
11. **Wait 5** seconds to see win/loss message
12. Go back to mini Adventure (type **5**)
13. Go to Relic Hunt (type **2** hit enter)
14. Select user 2 to play (type **2** hit enter)
15. Select Existing Character (type **1** hit enter)
16. **Repeat step 15** for user 2's character
17. Enter #'s 1-10 in this format ("**4 10**" enter) //There are 50 artifacts the user needs 5 to win
18. **Repeat 17** until one player wins
19. Wait **5** seconds to see the game end message

Retrospective

Basis for the Design:

The basis for the design was from Evan Stewart's A3. The reason we chose this version was due to the extensive work Evan did on A3 to make sure the refactored version was up to par. Other reasons that the team decided to use this as the base were the abstraction of the game Menus. This made it much easier for the new gamemodes to be added into the existing architecture, as everything was laid out in a readable and easily interpretable way.

High Points: Smooth integration of the game modes was the main highlight. The team had no issues creating the games, as the foundation was laid out nicely for others to interpret the basic design, reused from A3. The only hurdle for the game modes was the logic, but the integration was straightforward.

Low Points: The hardest part about this assignment was creating the logic for the game modes themselves. Despite it being easy to integrate the elements from the original game, into them having to then create the logic for how a game should operate, which was a challenge. After reflection between teammates, the development of the games went smoothly.

Major Challenges: Breaking up the work was a challenge for sure. As we all had different schedules, it was hard to work on it all at the same time, and to ensure no one stepped on each other's toes, we made sure to find an amicable split that would allow everyone to tackle a certain part of the development process. The solution for this was to divide the work up based on when people had time to work on it and create an ordering of how things needed to be developed. For this, we had Evan set up the GitHub repo with his old code, then he did some minor setup work to ensure that Calub and Omer could go and tackle a gamemode each. Archie was also on standby to help them with the integration, as well as make sure that we had everything integrated properly. Then afterwards Evan could come back and do testing of the game, ensuring that the team meets the requirements, as well as a source of quality control, as he made changes needed to make the games more playable (friendly to the testers)

AI Use

Our team did not utilize AI as it never said we had to in the assignment description. The choice for this was that it was simply not needed. The reasoning for this was that there was a clear framework that could be reused. Then the games themselves were straightforward to develop. At no point did we feel the need to use AI.

Justification of Quality (Patterns + Smells)

Patterns:

Non-Security Focused

Template Pattern: This can be seen from the templating of the menus through the use of the MenuTemplate.java class. This was used to avoid repeating similar menus.

Singleton Pattern: This is found in WorldClockClass.java. I made the world clock time need an instance to ensure the application enforces a single global clock.

Strategy Pattern: Within settings.java, the getFormattedTime Method uses a TimeDisplay enum to dynamically switch the time formatting when getFormattedTime is called. This allows the user to change the formatting of the time without having to restart the program or instantiate different user objects

Security-focused patterns

Fail-Safe Default Pattern:

Within Campaign.java, when it is created. The visibility status of a campaign is hardcoded to default to private. By defaulting to a restrictive access, it is a fail-safe by default.

Domain Driven Design DDD:

Based on the principles of DDD, my User class strictly defines a userID that is assigned at instantiation. Rather than relying on mutable attributes. This ensures a constant way to authenticate which user is playing.

Code Smells + Justification:

Magic Numbers:

This is present within the math logic for the time calculation. This was done to make the code more readable rather than declaring a variable just to use it 1x within a math equation

Reuse

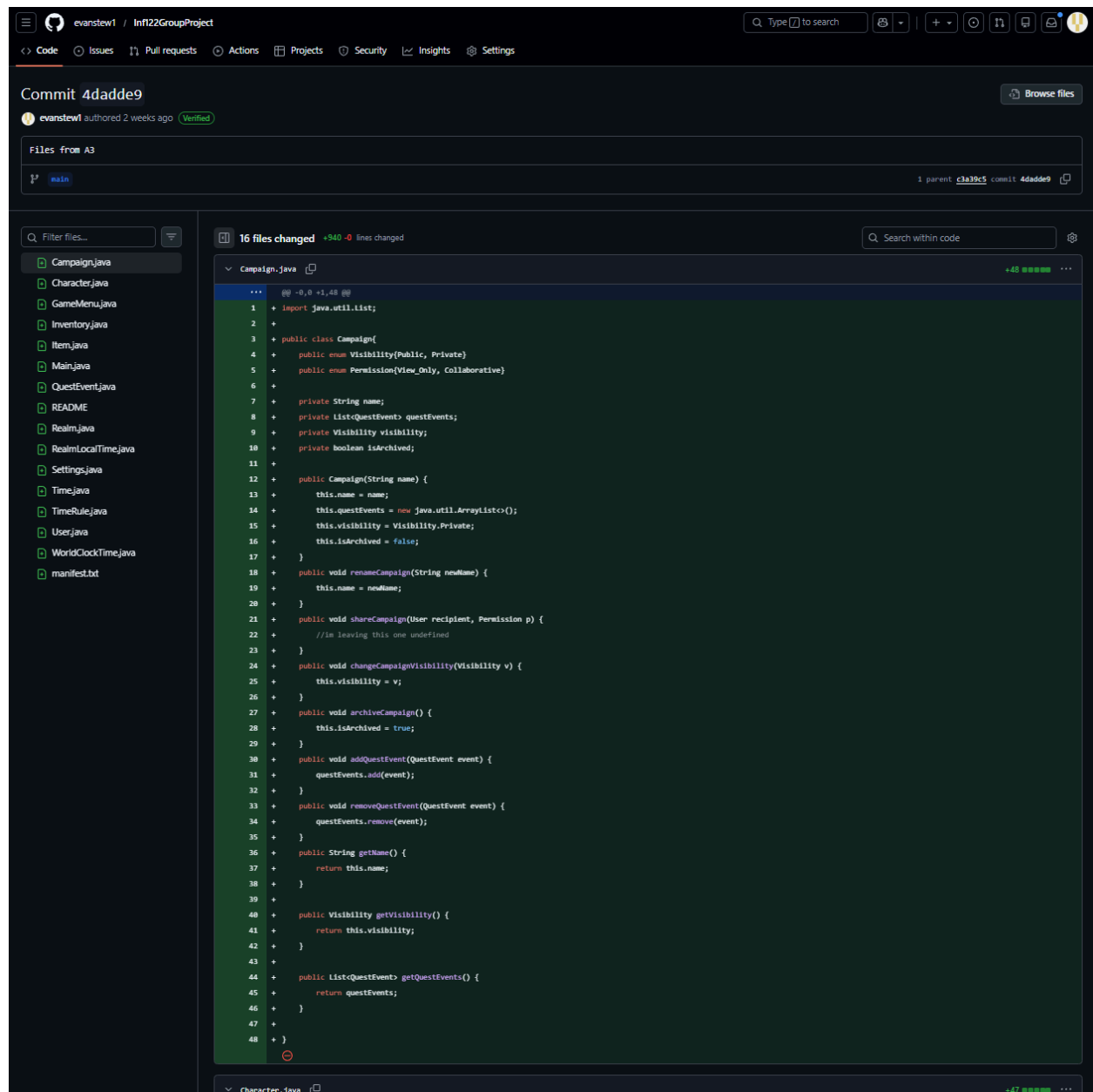
Source: It came from Evan Stewart's A3

Where in the Repo: It was the first commit to the Repo

What Changed: Game Menu was updated to include the running of the Mini games, the overall structure was unchanged

Where it's used: Used across the board to create a user/characters ect.

Evidence:



```
... @@ -0,0 +1,48 @@
1 + import java.util.List;
2 +
3 + public class Campaign{
4 +     public enum Visibility{Public, Private}
5 +     public enum Permission{View_Only, Collaborative}
6 +
7 +     private String name;
8 +     private List<QuestEvent> questEvents;
9 +     private Visibility visibility;
10 +     private boolean isArchived;
11 +
12 +     public Campaign(String name) {
13 +         this.name = name;
14 +         this.questEvents = new java.util.ArrayList<>();
15 +         this.visibility = Visibility.Private;
16 +         this.isArchived = false;
17 +     }
18 +     public void renameCampaign(String newName) {
19 +         this.name = newName;
20 +     }
21 +     public void shareCampaign(User recipient, Permission p) {
22 +         //leaving this one undefined
23 +     }
24 +     public void changeCampaignVisibility(Visibility v) {
25 +         this.visibility = v;
26 +     }
27 +     public void archiveCampaign() {
28 +         this.isArchived = true;
29 +     }
30 +     public void addQuestEvent(QuestEvent event) {
31 +         questEvents.add(event);
32 +     }
33 +     public void removeQuestEvent(QuestEvent event) {
34 +         questEvents.remove(event);
35 +     }
36 +     public String getName() {
37 +         return this.name;
38 +     }
39 +
40 +     public Visibility getVisibility() {
41 +         return this.visibility;
42 +     }
43 +
44 +     public List<QuestEvent> getQuestEvents() {
45 +         return questEvents;
46 +     }
47 +
48 + }
```